

BACCALAURÉAT GÉNÉRAL

Sujet 01 - Session 2026

NUMERIQUE ET SCIENCES INFORMATIQUES

ÉPREUVE DU SECOND GROUPE

DURÉE DE L'ÉPREUVE : **20 minutes**

DURÉE DE PRÉPARATION : **20 minutes**

L'utilisation d'une calculatrice est interdit

Le candidat préparera les différentes questions posées dans le sujet. Ces questions portent sur tout ou partie du programme du cycle terminale de la spécialité NSI.

Le candidat pourra répondre directement sur le sujet ou préparer les réponses aux questions sur une feuille de brouillon.

A la fin de la préparation, le candidat se présentera devant le jury muni de ce sujet ainsi que de toutes ses feuilles de brouillon.

Exercice 1 ►

Le nombre d'occurrences d'un caractère dans une chaîne de caractère est le nombre d'apparitions de ce caractère dans la chaîne.

- le nombre d'occurrences du caractère 'o' dans 'bonjour' est 2;
- le nombre d'occurrences du caractère 'b' dans 'Bébé' est 1;
- le nombre d'occurrences du caractère 'B' dans 'Bébé' est 1;
- le nombre d'occurrences du caractère ' ' dans 'Hello world !' est 2.

On cherche les occurrences des caractères dans une phrase. On souhaite stocker ces occurrences dans un dictionnaire dont les clefs seraient les caractères de la phrase et les valeurs l'occurrence de ces caractères.

Par exemple : avec la phrase 'Hello world !' le dictionnaire est le suivant :

```
{'H': 1, 'e': 1, 'l': 3, 'o': 2, ' ': 2, 'w': 1, 'r': 1, 'd': 1, '!': 1}
```

L'ordre des clefs n'a pas d'importance.

Écrire une fonction `nbr_occurrences` prenant comme paramètre une chaîne de caractères `chaine` et renvoyant le dictionnaire des nombres d'occurrences des caractères de cette chaîne.

</> Code Python

```
1 def nbr_occurrences(chaine):
2     ''' Renvoie un dictionnaire des occurrences de chaque caractère de la chaîne.
3     chaîne : une chaîne de caractères (str)
4     sortie : un dictionnaire
5     '''
6
7
8
9
10
11
12
13
14
15
16
17
18 assert nbr_occurrences("abracadabra") == {'a': 5, 'b': 2, 'r': 2, 'c': 1, 'd': 1}
```

Exercice 2 ►

Voici deux relations : LIVRES et AUTEURS.

- dans la table LIVRES, l'attribut id est la clé primaire,
- dans la table AUTEURS, l'attribut id est la clé primaire.
- dans la table LIVRES, l'attribut id_auteur est une clé étrangère qui fait référence à l'attribut id de la table AUTEURS.

id	titre	id_auteur	ann_publi	note
1	1984	1	1949	10
2	Dune	2	1965	8
3	Fondation	3	1951	9
4	Le meilleur des mondes	4	1931	9
5	Fahrenheit 451	5	1953	7
6	Ubik	6	1969	6
7	Chroniques martiennes	5	1950	7
8	La nuit des temps	7	1968	7
9	Blade Runner	6	1968	9
10	Les Robots	3	1950	10
11	La Planète des singes	8	1963	9
12	Ravage	7	1943	11
13	Le Maître du Haut Château	6	1962	7
14	La fin de l'éternité	3	1955	9

Relation : LIVRES

id	nom	prenom	ann_naissance	langue_ecriture
1	Orwell	George	1903	anglais
2	Herbert	Frank	1920	anglais
3	Azimov	Isaac	1920	anglais
4	Huxley	Aldous	1894	anglais
5	Bradbury	Ray	1920	anglais
6	K. Dick	Philip	1928	anglais
7	Barjavel	René	1911	français
8	Boule	Pierre	1912	français

Relation : AUTEURS

1. Avec les informations données sur ces relations, dire ce qu'affiche la requête suivante :

```

1 SELECT titre
2 FROM livres
3 WHERE note > 10;

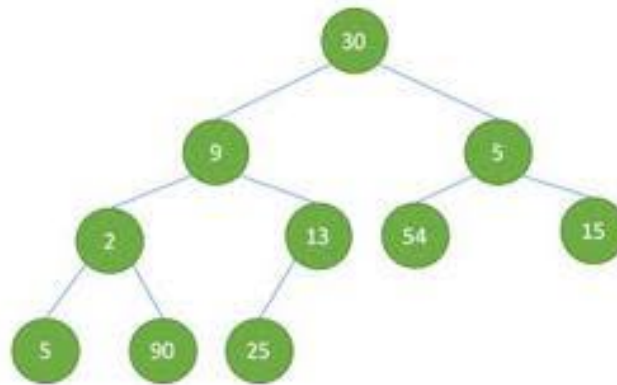
```

2. Ecrire une requête SQL permettant d'afficher le nom et le prénom de tous les auteurs.

3. Ecrire une requête SQL permettant d'afficher le titre et l'année de tous les livres écrits par l'auteur dont le nom est 'Hugo'.

Exercice 3 ►

On considère l'arbre binaire arb0 suivant :



1. Indiquer la taille de cet arbre. On rappelle que la taille d'un arbre vide est de 0

2. Est-ce un arbre binaire de recherche? Justifier.

3. Lister les valeurs de l'arbre arb0 parcouru de manière préfixe. On commencera le parcours par le côté gauche.

On désigne par SAG le sous-arbre gauche d'un noeud et par SAD le sous-arbre droit de ce noeud.

On rappelle aussi que la taille de l'arbre vérifie la propriété suivante :

$$\text{taille d'un arbre} = 1 + \text{taille}(\text{SAG}) + \text{taille}(\text{SAD})$$

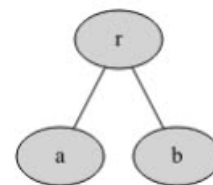
En python, un arbre peut être représenté à l'aide d'une liste de la manière suivante :

[valeur du noeud racine, SAG, SAD]

Un sous arbre vide sera donc noté par []

Ainsi l'arbre arb1 ci-contre est créé grâce à l'instruction :

```
arb1 = ['r', ['a', [], []], ['b', [], []]]
```



4. Compléter la fonction `taille_arbre` permettant de calculer la taille d'un arbre passé en paramètre.

</> Code Python

```

1 def taille_arbre(ar):
2     ''' Calcule la taille d'un arbre ar passe en parametre '''
3     if ar == []:
4         return ...
5     else:
6         return 1 + taille_arbre(.....) + taille_arbre(.....)
  
```